Elhaiba Hamza

Compiler & Low-Level Systems Engineer
github.com/codewithhippo17

Email: elhaiba.hamza@proton.me
Mobile: +212 6 69 84 21 71
Rabat, Morocco

Profile

Systems software engineer with deep expertise in C/C++, Linux internals, and low-level programming, with a strong interest in compiler design and toolchain development. Experienced in reading and reasoning about assembly code, understanding CPU architecture and ISAs, and working across the full software stack from bare-metal to user-space. Seeking to contribute to compiler infrastructure and AI hardware toolchains at the intersection of software and silicon.

Education

- **1337 Coding School – UM6P (42 Network)** — Morocco
  Systems-Level Software Engineering — *Sep 2021 – Jun 2023*
  - **Compiler & Language Tooling**: Implemented a custom shell (minishell) involving lexing, parsing, AST construction, and execution – directly applicable to compiler frontend design. Built a custom printf clone requiring deep understanding of format parsing and code generation patterns.
  - **Architecture & Low-Level Systems**: Completed projects covering CPU architecture concepts, memory management, IPC, signals, multithreading, and networking. Studied assembly-level behavior through GDB debugging and Valgrind analysis.
  - **C/C++ Mastery**: Extensive project work in C with strict norming and peer code review; introductory C++ and OOP; built custom data structure libraries (linked lists, hash maps, stacks) from scratch.
- **Faculty of Sciences and Techniques (FSTT)** — Morocco
  DEUST – Mathematics, Physics, and Engineering Sciences
  - **Foundation**: Studied advanced mathematics and physics, forming a strong analytical base for hardware architecture and compiler optimization theory.

Projects

- **minishell – Custom Unix Shell** — C
  Compiler Frontend Architecture — *2022*
  - **Lexer & Parser**: Implemented a full lexer (tokenizer) and recursive-descent parser producing an Abstract Syntax Tree (AST) – directly mirroring a compiler frontend pipeline.
  - **Execution Engine**: Evaluated the AST through a tree-walking interpreter handling built-ins, pipes, redirections, subshells, and signal propagation under POSIX semantics.
  - **Tooling**: Built and debugged entirely under Linux with GCC, Make, GDB, and Valgrind; enforced strict memory safety with zero leaks.
- **libft – Custom C Standard Library** — C
  Memory-Safe Systems Programming — *2021*
  - **Core Reimplementation**: Rebuilt memory, string, I/O, and linked-list primitives from scratch with no undefined behavior. Peer-reviewed and code-defended under rigorous 42 evaluation criteria.
- **ML Anomaly Detection Dashboard** — Python
  Taqathon Hackathon – Podium Finish — *2023*
  - **Real-Time Detection**: Built a sensor-data anomaly detection pipeline using ML models with a live dashboard; won podium placement at Taqathon.

Technical Skills

- **Languages**: C, C++, Python, Bash/Shell, english, french